M.Kumarasamy
College of Engineering
NAAC Accredited Autonomous Institution
Approved by AICTE & Affiliated to Anna University
ISO 9001:2015 Certified Institution
Thalavapalayam, Karur - 639 113, TAMILNADU.

A Project Report

on

PATIENT APPOINTMENT SCHEDULING SYSTEM

Submitted in partial fulfillment of requirements for the award of the course

of

EGB1221-DATABASE MANAGEMENT SYSTEMS

Under the guidance of

Dr.G.Subramaniam, Ph.D.,

Assistant Professor / EEE

Submitted By

KISHORE S **(927624BEE307)**

SANJAI B **(927624BEE084)**

SIBIDHARAN R **(927624BEE091)**

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

M.KUMARASAMY COLLEGE OF ENGINEERING
(Autonomous)

KARUR – 639 113

MAY 2026

M.Kumarasamy College of Engineering
NAAC Accredited Autonomous Institution
Approved by AICTE & Affiliated to Anna University
ISO 9001:2015 Certified Institution
Thalavapalayam, Karur - 639 113, TAMILNADU.

M. KUMARASAMY COLLEGE OF ENGINEERING

(Autonomous Institution affiliated to Anna University, Chennai)

KARUR – 639 113

BONAFIDE CERTIFICATE

This is to certify that this project report on **"PATIENT APPOINTMENT SCHEDULING SYSTEM"** is the bonafide work of **KISHORE S (927624BEE307), SANJAI B (927624BEE084), SIBIDHARAN R (927624BEE091)** who carried out the project work during the academic year 2025 - 2026 under my supervision.

Signature

Dr.G.SUBRAMANIAM, Ph.D.,

SUPERVISOR,

Department of Electrical and Electronics Engineering,

M.Kumarasamy College of Engineering,

Thalavapalayam, Karur -639 113.

Signature

Dr.J.UMA, M.E., Ph.D.,

HEAD OF THE DEPARTMENT,

Department of Electrical and Electronics Engineering,

M.Kumarasamy College of Engineering,

Thalavapalayam, Karur -639 113.

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

VISION OF THE INSTITUTION

To emerge as a leader among the top institutions in the field of technical education

MISSION OF THE INSTITUTION

- Produce smart technocrats with empirical knowledge who can surmount the global challenges
- Create a diverse, fully-engaged, learner-centric campus environment to provide quality education to the students
- Maintain mutually beneficial partnerships with our alumni, industry, and Professional associations

DEPARTMENT VISION, MISSION, PEO, PO AND PSO

VISION

To produce smart and dynamic professionals with profound theoretical and practical knowledge comparable with the best in the field.

MISSION

M1: Produce hi-tech professionals in the field of Electrical and Electronics Engineering by inculcating core knowledge

M2: Produce highly competent professionals with thrust on research.

M3: Provide personalized training to the students for enriching their skills.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- **PEO1:** Graduates will have flourishing career in the core areas of Electrical Engineering and allied disciplines.
- **PEO2:** Graduates will pursue higher studies and succeed in academic/research careers.
- **PEO3:** Graduates will be a successful entrepreneur in creating jobs related to Electrical and Electronics Engineering /allied disciplines.
- **PEO4:** Graduates will practice ethics and have habit of continuous learning for their success in the chosen career.

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment..

7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.
8. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
10. **Project management and finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11. **Life-long learning:** Recognize the need for, and have the preparation and ability for i) independent and life- long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change..

PROGRAM SPECIFIC OUTCOMES (PSOs)

- **PSO1:** Apply the basic concepts of mathematics and science to analyse and design circuits, controls, Electrical machines and drives to solve complex problems.
- **PSO2:** Apply relevant models, resources and emerging tools and techniques to provide solutions to power and energy related issues & challenges.
- **PSO3:** Design, Develop and implement methods and concepts to facilitate solutions for electrical and electronics engineering related real world problems.

ABSTRACT

The patient appointment sched application designed to simplify and automate the process of booking and managing medical appointments in hospitals or clinics. In traditional systems, appointment scheduling is often done manually, which can lead to errors, time delays, and difficulty in maintaining records. This project aims to overcome these issues by providing a digital platform for efficient appointment handling. The system allows patients to book appointments by entering essential details such as name, email, phone number, appointment date, time, doctor's name, and reason for visit. All the information entered by users is stored in a structured database using MySQL, ensuring proper organization and easy retrieval of data. The system also provides features to view, update, and delete appointment records, making it highly flexible and user-friendly This application is developed using front-end technologies for designing the user interface and backend integration with database management using phpMyAdmin. It demonstrates the practical implementation of Database Management System concepts such as data storage, retrieval, normalization, and CRUD operations. Overall, the system improves efficiency, reduces manual workload, and enhances the accuracy of appointment scheduling in healthcare environments.

ABSTRACT WITH POs AND PSOs MAPPING

ABSTRACT	COs MAPPED	POs MAPPED	PSOs MAPPED	SDG's
The Patient Appointment Scheduling System is a web-based application developed to simplify and automate the process of booking and managing medical appointments. Traditional manual systems often lead to errors, delays, and poor record maintenance. This system allows users to book appointments by entering details such as name, contact information, date, time, doctor, and reason for visit. All data is stored in a MySQL database, ensuring efficient storage, retrieval, and management. The system also provides options to view, update, and delete appointment records. Developed using front-end technologies and backend database integration through phpMyAdmin, the project demonstrates key Database Management System concepts such as data handling and CRUD operations. Overall, the system improves efficiency, reduces manual work, and enhances accuracy in appointment scheduling.	CO1 CO2 CO3 CO4 CO5	PO1 PO2 PO3 PO4 PO5 PO9 PO10 PO11	PSO2 PSO3	SDG 4 (Quality Education) , SDG 9 (Industry, Innovation and Infrastructure)

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	**vi**
1	**INTRODUCTION**	**1**
	1.1 OBJECTIVE	1
	1.2 OVERVIEW	2
	1.3 DATABASE MANAGEMENT SYSTEMS' CONCEPTS	3
2	**PROJECT METHODOLOGY**	4
	2.1 PROPOSED WORK	4
	2.2 ARCHITECTURE	5
	2.3 E-R DIAGRAM	8
3	**MODULE DESCRIPTION**	**11**
	3.1 Appointment Booking Module	11
	3.2 Appointment Management Module	12
	3.2.1 Update/Delete Module	13
	3.3 Database Module	13
	3.4 Admin Dashboard Module	14
4	**RESULTS AND DISCUSSION**	**16**
5	**CONCLUSION**	**21**
	REFERENCES	**22**
	APPENDIX	**23**

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE

The main objective of the Patient Appointment Scheduling System is to develop an efficient and user-friendly platform for managing medical appointments digitally. The system is designed to reduce the difficulties involved in traditional manual appointment booking methods followed in hospitals and clinics. Manual systems often consume more time, create confusion in maintaining records, and increase the chances of scheduling errors. To overcome these problems, the proposed system provides an automated solution that helps patients book appointments quickly and helps administrators manage records effectively.

The project mainly focuses on improving appointment handling by storing patient details in a structured MySQL database. The system allows users to add, view, update, and delete appointment records whenever required. It also improves communication between patients and hospital administrators by providing quick access to appointment information. The application demonstrates the practical implementation of Database Management System concepts such as CRUD operations, normalization, primary keys, and data retrieval techniques.

It aims to:

- Reduce manual work in hospitals
- Provide quick and efficient appointment booking
- Maintain patient records in a structured database
- Allow updating and deleting appointments

1.2 OVERVIEW

The Patient Appointment Scheduling System is a web-based application developed to simplify the process of scheduling and managing patient appointments in hospitals and clinics. The system provides a digital platform where patients can easily enter their details and book appointments with doctors without visiting the hospital physically. It replaces traditional manual methods that are often slow, inefficient, and difficult to manage.

The application is developed using front-end technologies such as HTML, CSS, and JavaScript for designing the user interface, while PHP is used for backend processing and MySQL is used for database management through phpMyAdmin. The system stores all appointment-related information in a structured database, ensuring proper organization, quick retrieval, and secure storage of records.

The system includes several modules such as Appointment Booking Module, Appointment Management Module, Update/Delete Module, Database Module, and Admin Dashboard Module. These modules work together to provide smooth appointment scheduling and management. Administrators can monitor all appointment records, update schedules, and delete unwanted data whenever necessary. The system improves reliability, reduces workload, and enhances the overall efficiency of healthcare appointment management.

Users can:

- Book new appointments
- View scheduled appointments
- Edit or delete existing bookings

The system uses a database to store all appointment details securely. It improves efficiency, accuracy, and accessibility compared to traditional manual booking methods.

1.3 DATABASE MANAGEMENT SYSTEMS' CONCEPTS

Database: A database is a collection of related data that is organized and stored systematically. In this project, it contains all the patient appointment records, making it easy to store, access, and manage information efficiently.

Table: A table is a structured format used to store data in rows and columns.

Primary Key: A primary key is a unique identifier for each record in a table. In this project, each appointment has a unique ID, which helps in identifying, updating, or deleting a specific appointment without confusion.

CRUD Operations: CRUD stands for Create, Read, Update, and Delete, which are the basic operations performed on the database. These operations help in managing the appointment data effectively.

Create → Add appointment: This operation allows users to enter and store new appointment details in the database.

Read → View appointments: This operation retrieves and displays the stored appointment records for users or administrators.

Update → Edit appointment: This operation enables modification of existing appointment details such as date, time, or doctor.

Delete → Remove appointment: This operation allows removal of unwanted or completed appointment records from the database.

Normalization: Normalization is the process of organizing data to reduce redundancy and improve data integrity. It ensures that the database is structured properly, avoids duplication, and maintains consistency in stored information.

CHAPTER 2

PROJECT METHODOLOGY

2.1 PROPOSED WORK

The proposed system is a Patient Appointment Scheduling System developed to automate and simplify the process of booking and managing medical appointments in hospitals or clinics. In the existing manual system, appointment scheduling is time-consuming, prone to human errors, and difficult to maintain records efficiently. To overcome these limitations, the proposed system provides a user-friendly digital platform that allows patients to book appointments easily from anywhere.

In this system, users can enter essential details such as patient name, email, phone number, appointment date, time, doctor's name, and reason for visit through a web-based interface. Once the data is submitted, it is stored in a structured MySQL database, ensuring proper organization and secure storage of information. The system is designed to handle large amounts of data efficiently and provides quick access to stored records.The proposed system also includes features for managing appointments, such as viewing all scheduled appointments, updating existing records, and deleting unwanted or completed appointments. These features help administrators maintain accurate and up-to-date information. The application is developed using front-end technologies for designing an interactive user interface and backend technologies for database connectivity and processing. It demonstrates the effective use of Database Management System concepts such as data storage, retrieval, normalization, and CRUD operations.

Overall, the proposed system aims to provide a reliable, efficient, and easy-to-use solution for appointment scheduling in healthcare environments, improving service quality and reducing waiting time for patients.

2.2 ARCHITECTURE

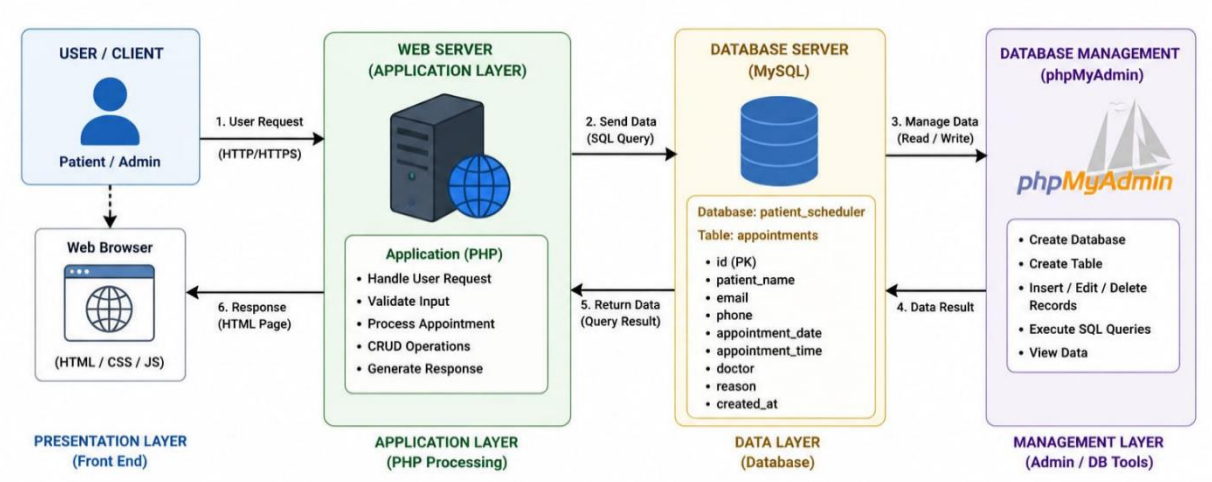


Figure 2.2.1: Architecture of Patient Appointment Scheduling System

The architecture of the Patient Appointment Scheduling System is designed to provide a simple, efficient, and user-friendly environment for managing medical appointments. The system follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Database Layer. This structure helps in maintaining proper communication between users, application processing, and database management. The architecture ensures smooth data flow, faster response time, secure data storage, and easy management of appointment records.

In this system, patients interact with the application through a web-based user interface developed using HTML, CSS, and JavaScript. The interface allows users to enter appointment details such as patient name, email, phone number, appointment date, appointment time, doctor's name, and reason for visit. The entered data is then sent to the backend application layer developed using PHP. The application layer validates the data, processes user requests, and performs CRUD (Create, Read, Update, Delete) operations. After processing, the information is stored in the MySQL database through phpMyAdmin.

The administrator can access the dashboard to monitor all appointments, edit records, delete completed appointments, and manage the entire scheduling process efficiently. The database layer securely stores all appointment-related information and enables quick retrieval of records whenever required. The architecture improves system reliability, minimizes manual work, and enhances the overall efficiency of hospital appointment management.

Components of Architecture

1. Presentation Layer

The presentation layer acts as the interface between the user and the system. It is responsible for collecting user inputs and displaying outputs in a user-friendly manner.

Functions:

- Provides appointment booking forms
- Displays appointment records
- Allows administrators to manage appointments
- Improves user interaction through a simple interface

Technologies Used:

- HTML
- CSS
- JavaScript

2. Application Layer

The application layer handles all processing activities of the system. It receives data from the user interface, validates it, and communicates with the database.

Functions:

- Processes appointment requests
- Performs CRUD operations

- Validates user input
- Handles database connectivity
- Controls system workflow

Technology Used:

- PHP

3. Database Layer

The database layer is the core component responsible for storing and managing appointment data. It maintains data consistency and supports quick retrieval of information.

Functions:

- Stores patient appointment details
- Maintains database records
- Supports data retrieval and updates
- Ensures data integrity and security

Technology Used:

- MySQL
- phpMyAdmin

Working Procedure of the System

1. User opens the appointment booking page.
2. Patient enters appointment details.
3. Data is validated by the application layer.
4. Valid information is stored in the MySQL database.
5. Administrator can view and manage all appointments.
6. Updated records are reflected immediately in the system.

2.3 E-R DIAGRAM

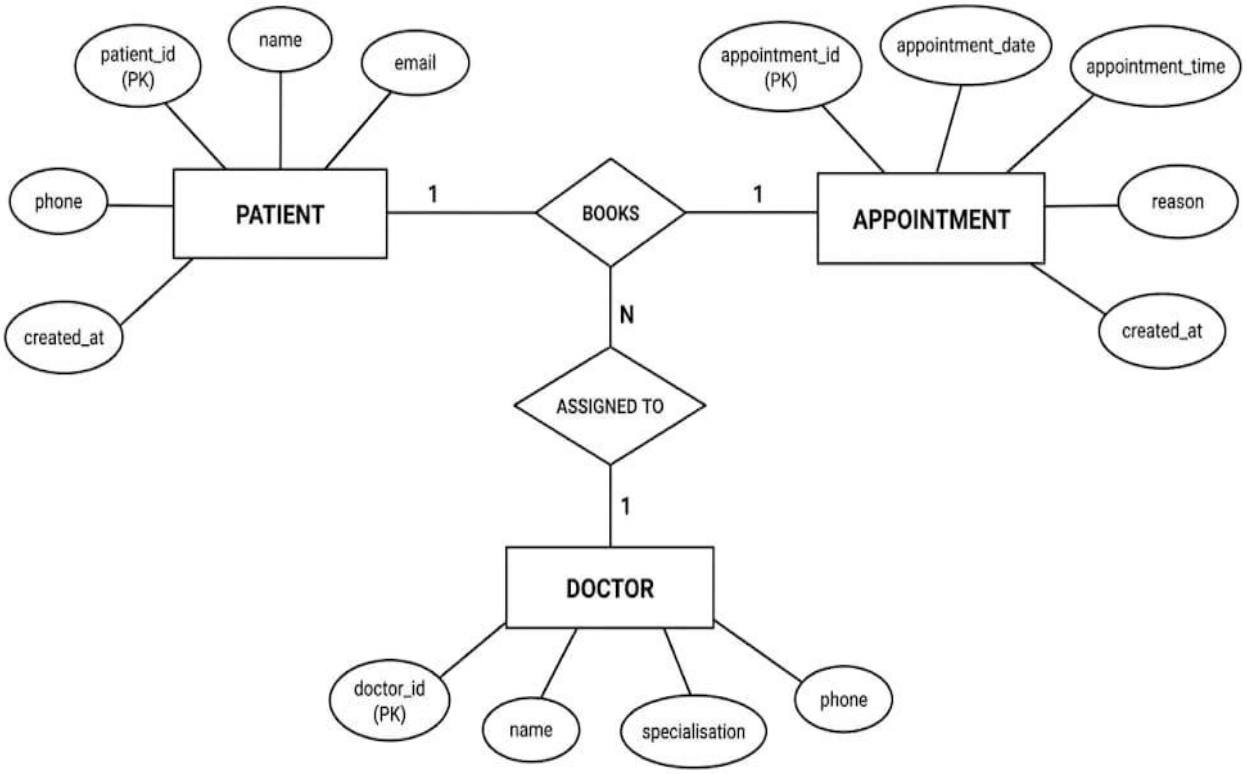


Figure 2.2.2: Patient Appointment Scheduling System

The Entity Relationship (E-R) Diagram represents the logical structure of the Patient Appointment Scheduling System database. It shows the entities involved in the system, their attributes, and the relationships between them. The E-R diagram helps in understanding how data is organized, stored, and connected within the database. It also plays an important role in database design and normalization.

In this project, the main entities are Patient, Doctor, and Appointment. Each entity contains specific attributes that describe the stored data. The Patient entity stores patient details such as patient ID, name, email, and phone number. The Doctor entity stores doctor-related information such as doctor ID, doctor name, and specialization. The Appointment entity contains appointment details including appointment ID, appointment date, appointment time, and reason for visit.

The Appointment entity acts as the connecting entity between Patient and Doctor. One patient can book multiple appointments, and one doctor can attend multiple patients. However, each appointment belongs to only one patient and one doctor. The relationships are maintained using foreign keys, which help in maintaining referential integrity in the database.

The E-R diagram improves database organization, reduces data redundancy, and ensures proper relationships between tables. It also supports efficient querying and easy maintenance of appointment records.

Entities and Attributes

1. Patient Entity

This entity stores patient-related information.

Attributes:

- Patient_ID (Primary Key)
- Patient_Name
- Email
- Phone_Number

Functions:

- Maintains patient details
- Identifies each patient uniquely
- Helps in appointment tracking

2. Doctor Entity

This entity stores doctor-related information.

Attributes:

- Doctor_ID (Primary Key)
- Doctor_Name
- Specialization

Functions:

- Stores doctor details
- Identifies available doctors
- Helps assign appointments

3. Appointment Entity

This entity stores appointment details.

Attributes:

- Appointment_ID (Primary Key)
- Appointment_Date
- Appointment_Time
- Reason_For_Visit
- Patient_ID (Foreign Key)
- Doctor_ID (Foreign Key)

Functions:

- Maintains appointment records
- Connects patient and doctor information
- Supports appointment management

Relationships in E-R Diagram

Patient – Appointment Relationship

- One patient can book many appointments.
- Each appointment belongs to one patient.
- Relationship Type: One-to-Many

Doctor – Appointment Relationship

- One doctor can handle multiple appointments.
- Each appointment is assigned to one doctor.
- Relationship Type: One-to-Many

CHAPTER 3

MODULE DESCRIPTION

The Patient Appointment Scheduling System is divided into different modules to ensure proper organization, easy management, and efficient functioning of the application. Each module performs a specific task and contributes to the overall operation of the system. The modular design helps in simplifying the development process, improving system performance, and making maintenance easier. The major modules included in the system are Appointment Booking Module, Appointment Management Module, Update/Delete Module, Database Module, and Admin Dashboard Module.

The modules are interconnected and work together to provide a smooth appointment scheduling experience for both patients and administrators. Each module is developed to handle a particular set of operations such as data entry, data storage, data modification, and data retrieval. The use of modules also improves flexibility, reduces system complexity, and allows future enhancements to be implemented easily.

3.1 Appointment Booking Module

The Appointment Booking Module is the primary module of the system that allows patients to schedule medical appointments easily through a web-based interface. This module is designed with a simple and user-friendly form where patients can enter their appointment details. The information entered by users is validated before being stored in the database to ensure accuracy and avoid incorrect entries. The module reduces the need for manual appointment registration and helps patients book appointments quickly without visiting the hospital directly. Once the patient submits the form, the system stores the details securely in the database and makes them available for administrator access.

This module improves efficiency, reduces waiting time, and provides a convenient method for appointment scheduling.

Allows users to enter:

- Patient Name
- Email Address
- Phone Number
- Appointment Date
- Appointment Time
- Doctor Name
- Reason for Visit

3.2 Appointment Management Module

The Appointment Management Module is responsible for managing all appointment records stored in the system. This module helps administrators view and organize appointment information efficiently. It retrieves stored data from the MySQL database and displays it in a structured format for easy access and monitoring.

The module ensures proper maintenance of appointment records and supports smooth communication between the application and database. It allows administrators to monitor daily appointments, verify patient details, and maintain accurate scheduling information. This module plays an important role in improving the reliability and effectiveness of the system.

It ensures proper storage, retrieval, and display of patient appointment information.

- Stores data in the database securely using structured tables
- Displays the list of all scheduled appointments in an organized format for easy view.

3.2.1 Update/Delete Module

The Update/Delete Module provides flexibility in managing appointment records. It allows administrators to modify existing appointment details whenever necessary and remove unwanted or completed records from the system. This module helps maintain accurate and up-to-date information in the database.

If there are changes in appointment schedules, doctor availability, or patient information, administrators can easily edit the records through this module. Similarly, invalid or completed appointments can be deleted to avoid unnecessary data accumulation. The module ensures that the database remains clean, organized, and efficient.

This module provides flexibility in managing existing appointment records by allowing modifications and removal of data when required.

- Edit appointment details such as patient name, date, time, doctor, and reason
- Delete unwanted, incorrect, or completed appointment records from the system
- Helps maintain accurate and up-to-date information in the database

3.3 Database Module

The Database Module is the core component of the Patient Appointment Scheduling System. It is responsible for storing, managing, and retrieving all appointment-related information using MySQL. This module ensures proper organization of records and maintains data integrity throughout the system.

The database is designed using structured tables with primary keys and appropriate fields for storing patient and appointment information. CRUD operations are performed through this module to add, retrieve, update, and delete data efficiently. The module also supports normalization techniques to minimize data redundancy and improve consistency.

The database module enables quick access to records and provides secure storage for all information entered into the system. It acts as the backbone of the application and plays a major role in overall system performance.

- Uses MySQL for storing and managing data
- Table: appointments
- Fields:
    - id – Unique identifier for each appointment (Primary Key)
    - patient_name – Name of the patient
    - email – Patient's email address for communication
    - phone – Contact number of the patient
    - appointment date – Scheduled date of appointment
    - appointment_time – Scheduled time of appointment
    - doctor – Assigned doctor for consultation
    - reason – Purpose of the visit
    - created_at – Timestamp indicating when the record was created

3.4 Admin Dashboard Module:

The Admin Dashboard Module is one of the most important modules in the Patient Appointment Scheduling System. It provides a centralized platform for administrators to monitor, control, and manage all appointment-related activities efficiently. This module is designed to simplify the administrative process by displaying all appointment records in an organized and user-friendly manner. Through the dashboard, administrators can easily access patient information, verify appointment schedules, update records, and remove unnecessary data whenever required.

The dashboard acts as the control center of the entire system. Instead of maintaining records manually in files or registers, administrators can manage all appointment information digitally through a single interface. This improves efficiency, reduces paperwork, and minimizes the chances of errors in appointment management. The module also ensures quick access to stored records, making the scheduling process faster and more reliable. The Admin Dashboard Module retrieves appointment data directly from the MySQL database and displays it in a tabular format for easy understanding. It helps administrators track appointments on a daily basis and maintain proper scheduling between doctors and patients. The module also supports update and delete operations, which helps in maintaining accurate and up-to-date information in the system.

Another important feature of this module is its ability to improve decision-making and overall hospital management. Administrators can monitor the number of appointments scheduled, identify completed appointments, and manage patient flow effectively. This reduces confusion and helps improve healthcare service quality. The dashboard interface is designed to be simple and easy to use so that administrators can operate the system without requiring advanced technical knowledge. It enhances productivity and reduces the workload involved in managing appointments manually.

The admin dashboard also helps in reducing management complexity by integrating all administrative functions into a single interface. It improves productivity and ensures better coordination in appointment scheduling activities.

- View all appointments in a single dashboard
- Manage records efficiently with options to update or delete data
- Provides better organization and quick access to appointment information

CHAPTER 4

RESULTS AND DISCUSSION

The Patient Appointment Scheduling System was successfully developed andtested using front-end technologies, PHP, and MySQL database management. The system achieved the main objective of simplifying and automating the appointment scheduling process in hospitals and clinics. The developed application provides a user-friendly environment where patients can easily book appointments and administrators can efficiently manage appointment records.

The system was tested with different appointment entries to verify the proper functioning of database connectivity, CRUD operations, and data retrieval processes. All modules worked successfully, including appointment booking, appointment management, update/delete operations, database storage, and admin dashboard functionalities. The application demonstrated accurate storage and retrieval of patient records without data loss or duplication.

The results obtained from the implementation show that the system reduces manual work, minimizes scheduling errors, and improves the efficiency of appointment handling. The use of MySQL ensured secure and structured data management, while the web-based interface improved accessibility and ease of use.

Overall, the system proved to be reliable, efficient, and suitable for healthcare appointment management, while also providing opportunities for future enhancements such as login authentication, notifications, and online consultation features.

1.Patient Appointment Booking Page

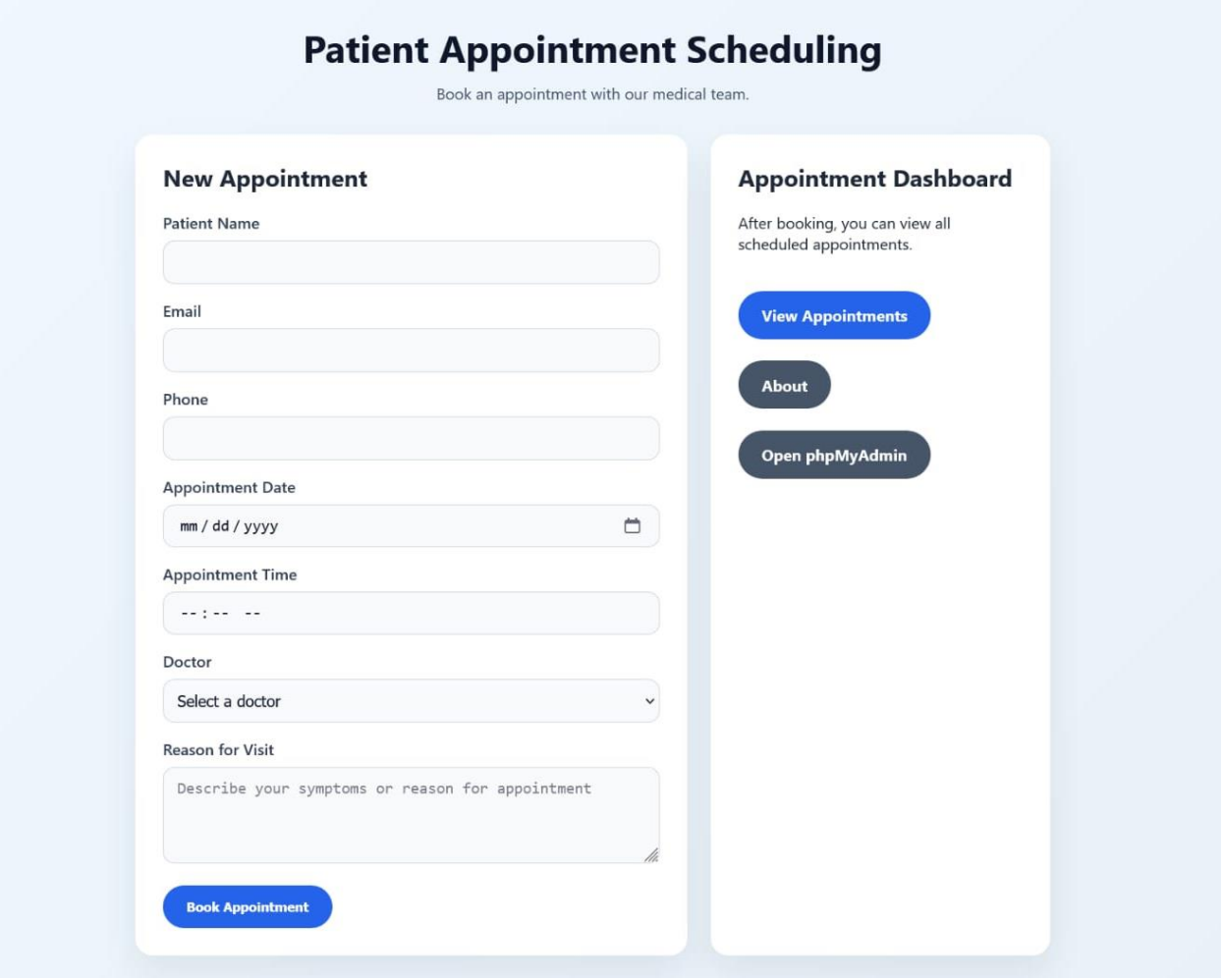


Figure 4.1 Patient Appointment Booking Page

The Patient Appointment Booking Page is the main interface through which patients can schedule appointments. The page contains a form where users enter their details such as patient name, email address, phone number, appointment date, appointment time, doctor name, and reason for visit.

The booking page was tested with multiple entries, and the system successfully stored all appointment information in the database. Input validation was also implemented to reduce incorrect or incomplete data entry. The interface was designed to be simple and user-friendly, allowing patients to book appointments quickly and efficiently.

2,Appointment Dashboard

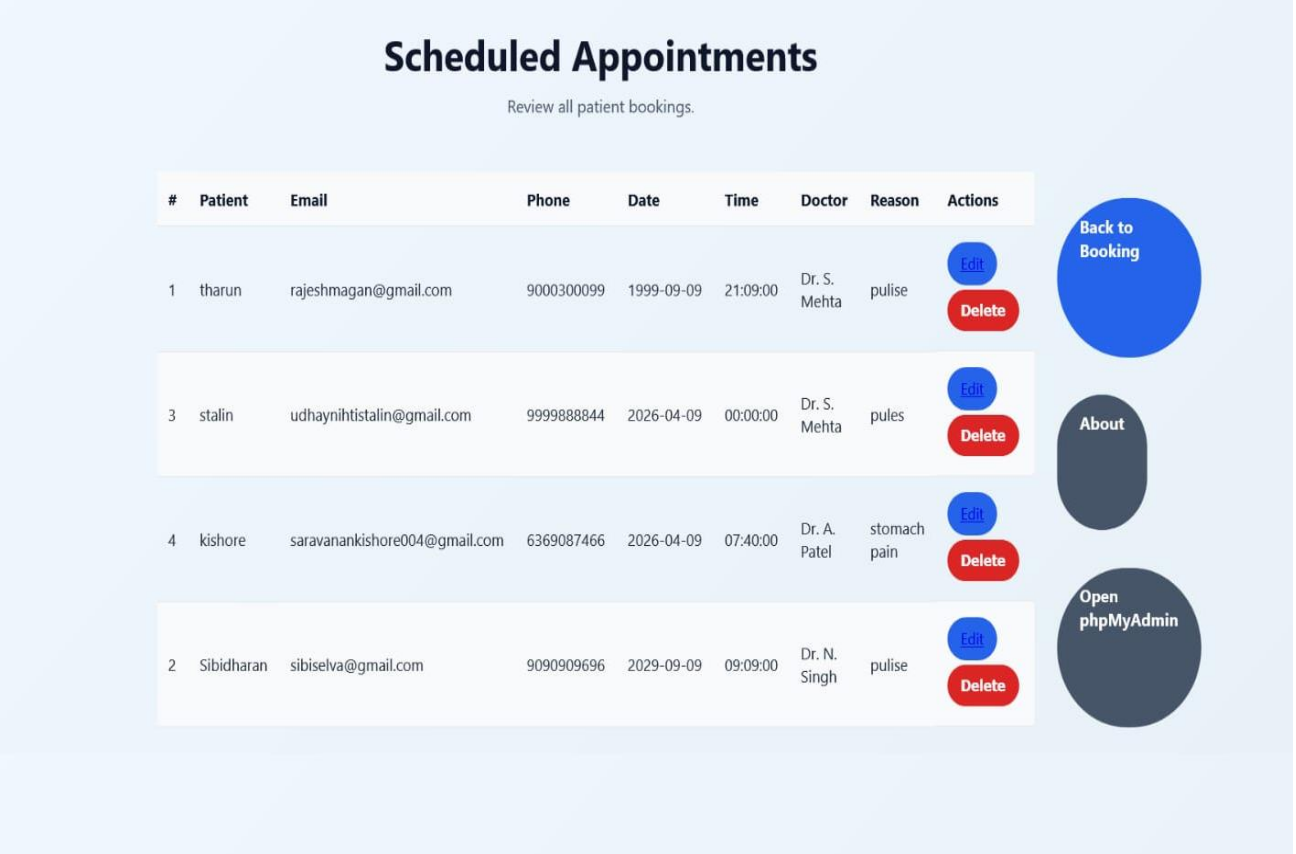


Figure 4.2 Appointment Dashboard

The Appointment Dashboard displays all scheduled appointments in a structured tabular format. This dashboard helps administrators monitor and manage patient appointments effectively. All records stored in the database are retrieved and displayed dynamically through the application.

The dashboard was tested for viewing, updating, and deleting records. The system successfully displayed appointment details and reflected modifications instantly after updates or deletions. This module improved administrative efficiency and reduced the complexity involved in managing appointment records manually.

3.phpMyAdmin Database Structure

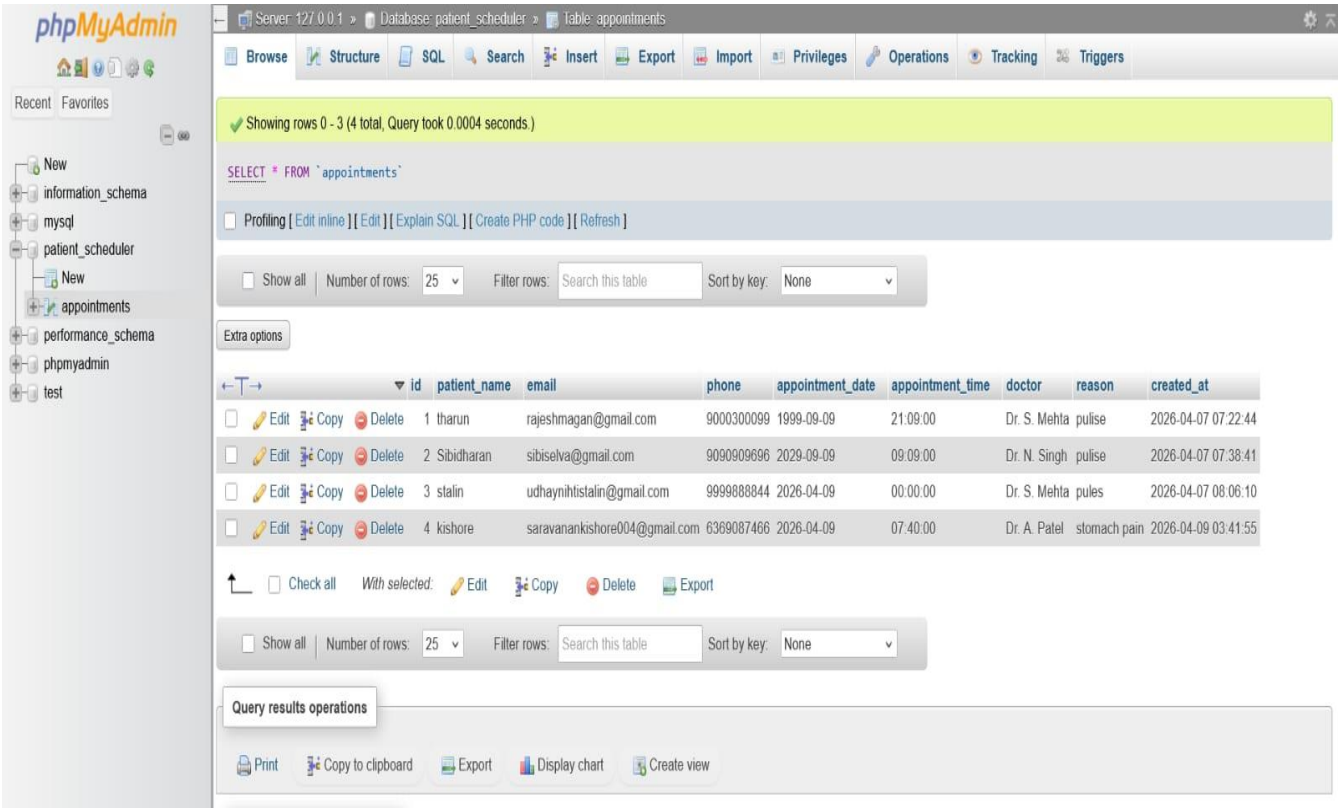


Figure 4.3 phpMyAdmin Database Structure

The database structure was created using MySQL through phpMyAdmin. The database contains a table named “appointments” that stores all appointment-related information. Proper field types, primary keys, and table structures were implemented to ensure efficient data management.

The database structure was tested for data insertion, retrieval, modification, and deletion operations. The system maintained data consistency and integrity throughout all operations. The use of normalization techniques reduced redundancy and improved database performance.

Results Observed

- Efficient database connectivity
- Accurate data storage
- Fast data retrieval
- Proper table management

4.Database Records

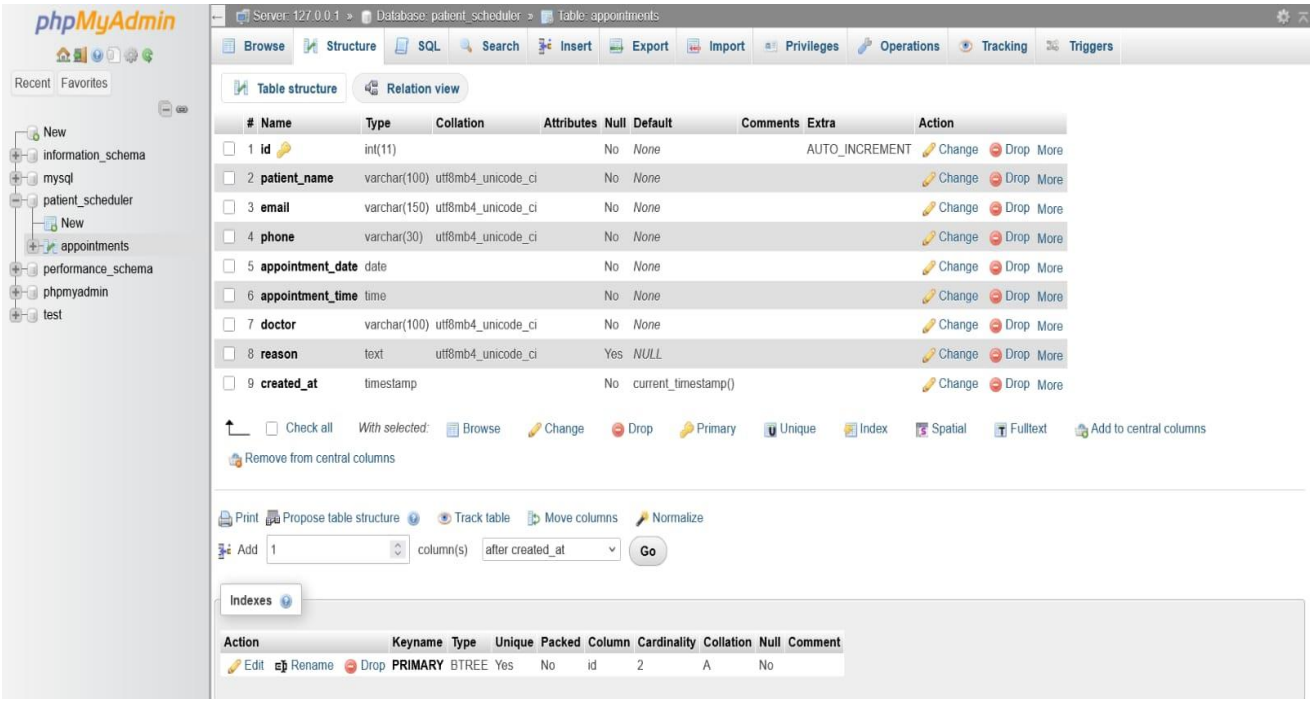


Figure 4.4 Database Records

The database records section shows the actual data stored after successful appointment booking. Each appointment entry was stored correctly with unique identification and timestamp information. The records confirmed that the system handled multiple entries efficiently without duplication or data corruption.

The retrieved records displayed proper synchronization between the application and database. Administrators were able to edit and delete records successfully, proving the reliability of CRUD functionality within the system.

Benefits:

- Maintains organized records
- Supports efficient appointment tracking
- Improves data reliability
- Simplifies information management

CHAPTER 5

CONCLUSION

The Patient Appointment Scheduling System successfully achieved its objective of simplifying and automating the process of managing medical appointments. The system reduced manual work involved in appointment scheduling and provided an efficient method for storing, retrieving, updating, and deleting appointment records. By using PHP and MySQL, the application ensured proper database management, data consistency, and secure storage of patient information. The project effectively demonstrated important Database Management System concepts such as CRUD operations, normalization, primary keys, and data retrieval. The developed modules worked efficiently and provided a user-friendly environment for both patients and administrators. The admin dashboard improved appointment monitoring and helped maintain organized records in the system. Overall, the system improved the accuracy, reliability, and efficiency of appointment management in healthcare environments.

REFERENCES

[1] Silberschatz, A., Korth, H.F., and Sudarshan, S., Database System Concepts, McGraw-Hill Education.

[2] Elmasri, R., and Navathe, S.B., Fundamentals of Database Systems, Pearson.

[3] MySQL Documentation, Available at: https://dev.mysql.com/doc/

[4] PHP Manual, Available at: https://www.php.net/manual/

[5] W3Schools, “HTML, CSS, and PHP Tutorials”, Available at: https://www.w3schools.com/

[6] Tutorials Point, “DBMS Tutorial”, Available at: https://www.tutorialspoint.com/dbms/

APPENDIX

(Coding)

DATABASE BACKEND:

Create Database Using SQL:

```
CREATE DATABASE IF NOT EXISTS patient_scheduler
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE patient_scheduler;

CREATE TABLE IF NOT EXISTS appointments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    phone VARCHAR(30) NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    doctor VARCHAR(100) NOT NULL,
    reason TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Database Connect Using PHP

```
<?php
$host = 'localhost';
$dbname = 'patient_scheduler';
$user = 'root';
$pass = '';
$charset = 'utf8mb4';
```

```
$dsn = "mysql:host=$host;charset=$charset";
$options = [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
    $pdo->exec("CREATE DATABASE IF NOT EXISTS `$dbname` CHARACTER
SET $charset COLLATE ${charset}_unicode_ci");
    $pdo->exec("USE `$dbname`");

    $pdo->exec(
        "CREATE TABLE IF NOT EXISTS appointments (
            id INT AUTO_INCREMENT PRIMARY KEY,
            patient_name VARCHAR(100) NOT NULL,
            email VARCHAR(150) NOT NULL,
            phone VARCHAR(30) NOT NULL,
            appointment_date DATE NOT NULL,
            appointment_time TIME NOT NULL,
            doctor VARCHAR(100) NOT NULL,
            reason TEXT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )"
    );
}
```

```
catch (PDOException $e) {
    die('Database connection failed: ' . htmlspecialchars($e->getMessage(),
ENT_QUOTES, 'UTF-8'));
}
```

Edit Appointment Using PHP:

```
<?php
require_once 'db_connect.php';

$appointmentId = isset($_GET['id']) ? (int) $_GET['id'] : 0;
if ($appointmentId <= 0) {
    header('Location: view_appointments.php');
    exit;
}

$sql = 'SELECT id, patient_name, email, phone, appointment_date,
appointment_time, doctor, reason FROM appointments WHERE id = :id';
$stmt = $pdo->prepare($sql);
$stmt->execute([':id' => $appointmentId]);
$appointment = $stmt->fetch();

if (!$appointment) {
    header('Location: view_appointments.php');
    exit;
}
?>
```

Submit Appointment Using PHP:

```
<?php
require_once 'db_connect.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: index.php');
    exit;
}

$patientName = trim($_POST['patient_name'] ?? '');
$email = trim($_POST['email'] ?? '');
$phone = trim($_POST['phone'] ?? '');
$appointmentDate = trim($_POST['appointment_date'] ?? '');
$appointmentTime = trim($_POST['appointment_time'] ?? '');
$doctor = trim($_POST['doctor'] ?? '');
$reason = trim($_POST['reason'] ?? '');
$appointmentId = isset($_POST['appointment_id']) ? (int)
$_POST['appointment_id'] : 0;
$errors = [];
if ($patientName === '') {
    $errors[] = 'Patient name is required.';
}
if ($email === '' || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors[] = 'A valid email address is required.';
}
if ($phone === '') {
    $errors[] = 'Phone number is required.';
}
```

```
if ($appointmentDate === '') {
    $errors[] = 'Appointment date is required.';
}
if ($appointmentTime === '') {
    $errors[] = 'Appointment time is required.';
}
if ($doctor === '') {
    $errors[] = 'Please select a doctor.';
}

if (!empty($errors)) {
    $message = implode('<br>', array_map('htmlspecialchars', $errors));
    $status = 'error';
} else {
    if ($appointmentId > 0) {
        $sql = 'UPDATE appointments SET patient_name = :patient_name, email =
:email, phone = :phone, appointment_date = :appointment_date, appointment_time
= :appointment_time, doctor = :doctor, reason = :reason WHERE id = :id';
        $stmt = $pdo->prepare($sql);
        $params = [
            ':patient_name' => $patientName,
            ':email' => $email,
            ':phone' => $phone,
            ':appointment_date' => $appointmentDate,
            ':appointment_time' => $appointmentTime,
            ':doctor' => $doctor,
            ':reason' => $reason,
            ':id' => $appointmentId, ];
$messageText = 'Appointment successfully updated.';
```

```
    } else {
        $sql = 'INSERT INTO appointments (patient_name, email, phone,
appointment_date, appointment_time, doctor, reason) VALUES (:patient_name,
:email, :phone, :appointment_date, :appointment_time, :doctor, :reason)';
        $stmt = $pdo->prepare($sql);
        $params = [
            ':patient_name' => $patientName,
            ':email' => $email,
            ':phone' => $phone,
            ':appointment_date' => $appointmentDate,
            ':appointment_time' => $appointmentTime,
            ':doctor' => $doctor,
            ':reason' => $reason,
        ];
        $messageText = 'Appointment successfully booked.';
    }
    try {
        $stmt->execute($params);
        $message = $messageText;
        $status = 'success';
    } catch (PDOException $e) {
        $message = 'Unable to save appointment: ' . htmlspecialchars($e-
>getMessage(), ENT_QUOTES, 'UTF-8');
        $status = 'error';
    }
}
?>
```

View Appointment Using PHP

```
<?php
require_once 'db_connect.php';

$sql = 'SELECT id, patient_name, email, phone, appointment_date,
appointment_time, doctor, reason, created_at FROM appointments ORDER BY
appointment_date ASC, appointment_time ASC';
$stmt = $pdo->query($sql);
$appointments = $stmt->fetchAll();
?>
```

Delete Appointment Using PHP:

```
<?php
require_once 'db_connect.php';
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: view_appointments.php');
    exit;
}
$appointmentId = isset($_POST['id']) ? (int) $_POST['id'] : 0;
if ($appointmentId <= 0) {
    header('Location: view_appointments.php');
    exit;
}
$sql = 'DELETE FROM appointments WHERE id = :id';
$stmt = $pdo->prepare($sql);
$stmt->execute([':id' => $appointmentId]);
header('Location: view_appointments.php');
exit;
```

FRONTEND USING HTML:

Booking Page Using HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About Patient Scheduler</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>About This Appointment Scheduler</h1>
            <p>This system manages patient appointments with a simple PHP frontend
and MySQL backend.</p>
        </header>
        <main>
            <section class="form-card">
                <h2>System Features</h2>
                <ul>
                    <li>Book new patient appointments</li>
                    <li>View scheduled appointments</li>
                    <li>Edit appointment details</li>
                    <li>Delete appointments</li>
                    <li>Automatic database and table creation</li>
                </ul>
            </section>
```

```
        <section class="info-card">
            <h2>Developer</h2>
            <p>This project is assembled by an assistant to help you quickly run a working appointment scheduling web application.</p>
            <p>Use the links below to return to the app or open phpMyAdmin.</p>
            <div class="actions">
                <a class="button-link" href="index.php">Back to Booking</a>
                <a class="button-link button-secondary" href="http://localhost/phpmyadmin" target="_blank" rel="noopener">Open phpMyAdmin</a>
            </div>
        </section>
    </main>
  </div>
</body>
</html>
```

Home Index Using HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Patient Appointment Scheduling</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
```

```
<header>
  <h1>Patient Appointment Scheduling</h1>
  <p>Book an appointment with our medical team.</p>
</header>

<main>
  <section class="form-card">
    <h2>New Appointment</h2>
    <form action="submit_appointment.php" method="post">
      <label for="patient_name">Patient Name</label>
      <input type="text" id="patient_name" name="patient_name" required>

      <label for="email">Email</label>
      <input type="email" id="email" name="email" required>

      <label for="phone">Phone</label>
      <input type="tel" id="phone" name="phone" required>

      <label for="appointment_date">Appointment Date</label>
      <input type="date" id="appointment_date" name="appointment_date"
required>

      <label for="appointment_time">Appointment Time</label>
      <input type="time" id="appointment_time" name="appointment_time"
required>

      <label for="doctor">Doctor</label>
      <select id="doctor" name="doctor" required>
```

```
                <option value="">Select a doctor</option>
                <option value="Dr. A. Patel">Dr. A. Patel</option>
                <option value="Dr. S. Mehta">Dr. S. Mehta</option>
                <option value="Dr. N. Singh">Dr. N. Singh</option>
                <option value="Dr. R. Kumar">Dr. R. Kumar</option>
            </select>
            <label for="reason">Reason for Visit</label>
            <textarea id="reason" name="reason" rows="4" placeholder="Describe
your symptoms or reason for appointment"></textarea>
            <button type="submit">Book Appointment</button>
        </form>
    </section>

    <section class="info-card">
        <h2>Appointment Dashboard</h2>
        <p>After booking, you can view all scheduled appointments.</p>
        <a class="button-link" href="view_appointments.php">View
Appointments</a>
        <a class="button-link button-secondary" href="about.php">About</a>
        <a class="button-link button-secondary"
href="http://localhost/phpmyadmin" target="_blank" rel="noopener">Open
phpMyAdmin</a>
    </section>
  </main>
 </div>
</body>
</html>
```

Front End For Edit Appointment:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Edit Appointment</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>Edit Appointment</h1>
            <p>Update the patient appointment details below.</p>
        </header>

        <main>
            <section class="form-card">
                <h2>Appointment Details</h2>
                <form action="submit_appointment.php" method="post">
                    <input type="hidden" name="appointment_id" value="<?php echo
htmlspecialchars($appointment['id'], ENT_QUOTES, 'UTF-8'); ?>">

                    <label for="patient_name">Patient Name</label>
                    <input type="text" id="patient_name" name="patient_name"
value="<?php echo htmlspecialchars($appointment['patient_name'], ENT_QUOTES,
'UTF-8'); ?>" required>
```

```
            <label for="email">Email</label>
            <input type="email" id="email" name="email" value="<?php echo
htmlspecialchars($appointment['email'], ENT_QUOTES, 'UTF-8'); ?>" required>

            <label for="phone">Phone</label>
            <input type="tel" id="phone" name="phone" value="<?php echo
htmlspecialchars($appointment['phone'], ENT_QUOTES, 'UTF-8'); ?>" required>

            <label for="appointment_date">Appointment Date</label>
            <input type="date" id="appointment_date" name="appointment_date"
value="<?php echo htmlspecialchars($appointment['appointment_date'],
ENT_QUOTES, 'UTF-8'); ?>" required>

            <label for="appointment_time">Appointment Time</label>
            <input type="time" id="appointment_time" name="appointment_time"
value="<?php echo htmlspecialchars($appointment['appointment_time'],
ENT_QUOTES, 'UTF-8'); ?>" required>

            <label for="doctor">Doctor</label>
            <select id="doctor" name="doctor" required>
                <option value="">Select a doctor</option>
                <option value="Dr. A. Patel" <?php echo $appointment['doctor'] ===
'Dr. A. Patel' ? 'selected' : ''; ?>>Dr. A. Patel</option>
                <option value="Dr. S. Mehta" <?php echo $appointment['doctor'] ===
'Dr. S. Mehta' ? 'selected' : ''; ?>>Dr. S. Mehta</option>
                <option value="Dr. N. Singh" <?php echo $appointment['doctor'] ===
'Dr. N. Singh' ? 'selected' : ''; ?>>Dr. N. Singh</option>
```

```
                    <option value="Dr. R. Kumar" <?php echo $appointment['doctor']
=== 'Dr. R. Kumar' ? 'selected' : ''; ?>>Dr. R. Kumar</option>
                </select>

                <label for="reason">Reason for Visit</label>
                <textarea id="reason" name="reason" rows="4"><?php echo
htmlspecialchars($appointment['reason'], ENT_QUOTES, 'UTF-8'); ?></textarea>

                <button type="submit">Save Changes</button>
            </form>
            <div class="actions">
                <a class="button-link" href="view_appointments.php">Cancel</a>
            </div>
        </section>
    </main>
  </div>
</body>
</html>
```

Front End For Submit Appointment:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Appointment Result</title>
  <link rel="stylesheet" href="style.css">
</head>
```

```
<body>
    <div class="container">
        <header>
            <h1>Appointment Result</h1>
        </header>
        <main>
            <div class="message-card <?php echo $status; ?>">
                <p><?php echo $message; ?></p>
            </div>
            <div class="actions">
                <a class="button-link" href="index.php">Book Another Appointment</a>
                <a class="button-link" href="view_appointments.php">View
Appointments</a>
            </div>
        </main>
    </div>
</body>
</html>
```

Front End For View Appointment:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>View Appointments</title>
    <link rel="stylesheet" href="style.css">
</head>
```

```
<body>
  <div class="container">
    <header>
      <h1>Scheduled Appointments</h1>
      <p>Review all patient bookings.</p>
    </header>
    <main>
      <?php if (empty($appointments)): ?>
        <div class="message-card info">
          <p>No appointments have been scheduled yet.</p>
        </div>
      <?php else: ?>
        <table>
          <thead>
            <tr>
              <th>#</th>
              <th>Patient</th>
              <th>Email</th>
              <th>Phone</th>
              <th>Date</th>
              <th>Time</th>
              <th>Doctor</th>
              <th>Reason</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody>
            <?php foreach ($appointments as $appointment): ?>
```

```
                <tr>
                    <td><?php echo htmlspecialchars($appointment['id'],
ENT_QUOTES, 'UTF-8'); ?></td>
                    <td><?php echo htmlspecialchars($appointment['patient_name'],
ENT_QUOTES, 'UTF-8'); ?></td>
                    <td><?php echo htmlspecialchars($appointment['email'],
ENT_QUOTES, 'UTF-8'); ?></td>
                    <td><?php echo htmlspecialchars($appointment['phone'],
ENT_QUOTES, 'UTF-8'); ?></td>
                    <td><?php echo
htmlspecialchars($appointment['appointment_date'], ENT_QUOTES, 'UTF-8');
?></td>
                    <td><?php echo
htmlspecialchars($appointment['appointment_time'], ENT_QUOTES, 'UTF-8');
?></td>
                    <td><?php echo htmlspecialchars($appointment['doctor'],
ENT_QUOTES, 'UTF-8'); ?></td>
                    <td><?php echo nl2br(htmlspecialchars($appointment['reason'],
ENT_QUOTES, 'UTF-8')); ?></td>
                    <td class="actions-cell">
                        <a class="button-small"
href="edit_appointment.php?id=<?php echo urlencode($appointment['id']);
?>">Edit</a>
                        <form action="delete_appointment.php" method="post"
onsubmit="return confirm('Delete this appointment?');">
                            <input type="hidden" name="id" value="<?php echo
htmlspecialchars($appointment['id'], ENT_QUOTES, 'UTF-8'); ?>">
                            <button type="submit" class="button-small button-
```

```
delete">Delete</button>
                        </form>
                    </td>
                </tr>
            <?php endforeach; ?>
        </tbody>
    </table>
<?php endif; ?>
<div class="actions">
    <a class="button-link" href="index.php">Back to Booking</a>
    <a class="button-link button-secondary" href="about.php">About</a>
    <a class="button-link button-secondary"
href="http://localhost/phpmyadmin" target="_blank" rel="noopener">Open
phpMyAdmin</a>
</div>
</main>
</div>
</body>
</html>
```

Front End Theme Using CSS:

```
* {
  box-sizing: border-box;
}
body {
  margin: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #f0f7ff 0%, #e8f1f8 100%);
  color: #1f2937;
```

```
}
.container {
    max-width: 980px;
    margin: 0 auto;
    padding: 24px;
}
header {
    text-align: center;
    margin-bottom: 32px;
}
header h1 {
    margin: 0;
    font-size: 2.4rem;
    color: #0f172a;
}
header p {
    margin: 10px auto 0;
    max-width: 640px;
    color: #475569;
}
main {
    display: grid;
    grid-template-columns: 1.3fr 0.8fr;
    gap: 24px;
}


.form-card,
```

```
.info-card,
.message-card {
    background: #ffffff;
    border-radius: 18px;
    box-shadow: 0 18px 40px rgba(15, 23, 42, 0.08);
    padding: 28px;
}
.form-card h2,
.info-card h2 {
    margin-top: 0;
    font-size: 1.5rem;
}
label {
    display: block;
    margin-top: 16px;
    font-weight: 600;
    color: #334155;
}
input[type="text"],
input[type="email"],
input[type="tel"],
input[type="date"],
input[type="time"],
select,
textarea {
    width: 100%;
    margin-top: 8px;
    padding: 12px 14px;
```

```
    border: 1px solid #cbd5e1;
    border-radius: 12px;
    font-size: 1rem;
    background: #f8fafc;
    color: #0f172a;
}
textarea {
    resize: vertical;
}
button,
.button-link {
    display: inline-block;
    margin-top: 22px;
    padding: 14px 24px;
    border: none;
    border-radius: 999px;
    background: #2563eb;
    color: #ffffff;
    text-decoration: none;
    font-weight: 700;
    cursor: pointer;
    transition: transform 0.2s ease, background-color 0.2s ease;
}
.button-secondary {
    background: #475569;
}
.button-secondary:hover {
    background: #334155;}
```

```
.button-small {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    padding: 10px 14px;
    margin: 0 4px 4px 0;
    border-radius: 999px;
    background: #2563eb;
    font-size: 0.9rem;
}
.button-delete {
    background: #dc2626;
}
.actions-cell {
    display: flex;
    flex-wrap: wrap;
    align-items: center;
}
.actions-cell form {
    margin: 0;
}
button:hover,
.button-link:hover,
.button-small:hover {
    background: #1d4ed8;
    transform: translateY(-1px);
}
.button-delete:hover {
```

```
    background: #b91c1c;
}
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 18px;
}
th,
td {
    padding: 14px 12px;
    border-bottom: 1px solid #e2e8f0;
    text-align: left;
    font-size: 0.95rem;
}
th {
    color: #0f172a;
    background: #f8fafc;
}
tr:nth-child(even) {
    background: #f8fafc;
}
.message-card {
    border-left: 5px solid #2563eb;
}
.message-card.success {
    border-color: #16a34a;
}
.message-card.error {
```

```
    border-color: #dc2626;
}
.message-card.info {
    border-color: #2563eb;
}
.actions {
    margin-top: 20px;
    display: flex;
    gap: 12px;
    flex-wrap: wrap;
}
@media (max-width: 900px) {
    main {
        grid-template-columns: 1fr;
    }
}
```